

Yes. The best way to learn Postman comprehensively is to build **one complete REST API project** that deliberately exposes different HTTP methods, parameter types, request bodies, headers, cookies, authentication, file uploads, validation, pagination, sorting, filtering, JSON, XML, form data, and error responses.

I'll use **Spring Boot 4.1.0 + Java 25**. As of August 18, 2026, Spring Boot 4.1.0 is the latest stable Spring Boot release, Spring Framework 7.0.8 is stable, and Java 25 is the latest LTS release. [Home](#) Spring Boot 4.1.0's documentation also recommends the newer spring-boot-starter-webmvc starter for Spring MVC applications, although spring-boot-starter-web remains available. [Home](#)

1. What we are going to build

Project name:

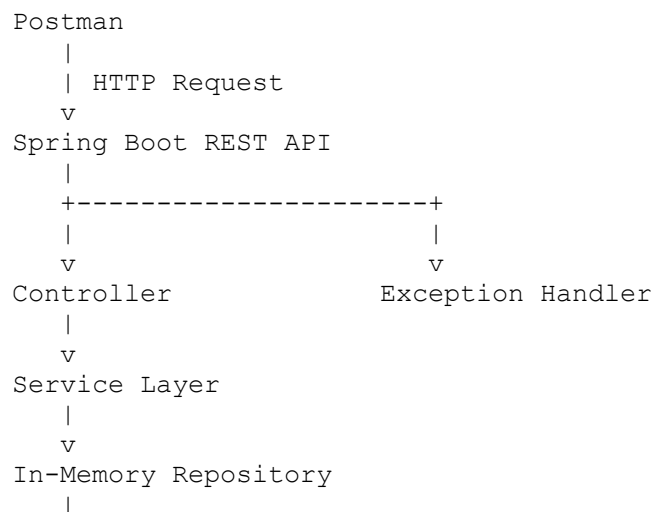
Postman API Practice Lab

It will demonstrate:

#	API concept	Example
1	GET	Retrieve products
2	POST	Create product
3	PUT	Replace product
4	PATCH	Partially update product
5	DELETE	Delete product
6	Path Variable	/products/101
7	Query Parameter	?category=Electronics
8	Multiple Query Parameters	?page=0&size=5&sortBy=price
9	Request Header	X-API-KEY
10	Authorization Header	Bearer ...
11	JSON Body	Content-Type: application/json
12	XML Body	Content-Type: application/xml
13	Form URL Encoded	application/x-www-form-urlencoded
14	Multipart	File + JSON/text fields
15	Cookie	Cookie: clientId=...

16	Response Headers	X-Request-ID
17	Status Codes	200, 201, 204, 400, 401, 404, 409
18	Validation	@NotBlank, @Positive
19	Exception Handling	Centralized error response
20	Pagination	page, size
21	Sorting	sortBy, direction
22	Filtering	category/min/max price
23	Authentication demo	Login/token
24	Content negotiation	JSON/XML
25	Plain text	text/plain
26	Health check	/api/v1/health
27	Request echo	Inspect headers/parameters
28	File upload	multipart/form-data
29	API versioning	/api/v1/...
30	Postman environment variables	{{baseUrl}}

2. Architecture



This project intentionally uses an **in-memory data store** rather than MySQL so that you can concentrate on Postman and REST APIs first.

3. Complete project structure

```
postman-api-practice-lab/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── example/
│   │   │   │   │   ├── postmanlab/
│   │   │   │   │   │   ├── PostmanApiPracticeLabApplication.java
│   │   │   │   │   │   ├── controller/
│   │   │   │   │   │   │   ├── AuthController.java
│   │   │   │   │   │   │   ├── ProductController.java
│   │   │   │   │   │   │   └── UtilityController.java
│   │   │   │   │   │   ├── dto/
│   │   │   │   │   │   │   ├── LoginRequest.java
│   │   │   │   │   │   │   ├── LoginResponse.java
│   │   │   │   │   │   │   ├── ProductRequest.java
│   │   │   │   │   │   │   ├── ProductPatchRequest.java
│   │   │   │   │   │   │   └── ProductResponse.java
│   │   │   │   │   │   ├── exception/
│   │   │   │   │   │   │   ├── ApiError.java
│   │   │   │   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   │   │   │   └── ProductNotFoundException.java
│   │   │   │   │   │   ├── model/
│   │   │   │   │   │   │   └── Product.java
│   │   │   │   │   │   └── service/
│   │   │   │   │   │       ├── AuthService.java
│   │   │   │   │   │       └── ProductService.java
│   │   │   │   │   └── resources/
│   │   │   │   │       └── application.properties
│   │   └── test/
│   │   │   ├── java/
│   │   │   │   ├── com/
│   │   │   │   │   ├── example/
│   │   │   │   │   │   └── postmanlab/
```

4. pom.xml

Spring Boot's dependency management lets us omit versions for dependencies managed by Spring Boot. [Home](#)

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.0</version>
    <relativePath/>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>postman-api-practice-lab</artifactId>
  <version>1.0.0</version>

  <name>postman-api-practice-lab</name>
  <description>
    Complete REST API project for learning and testing APIs using Postman
  </description>

  <properties>
    <java.version>25</java.version>
  </properties>

  <dependencies>

    <!-- Spring MVC / REST -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <!-- Bean Validation -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- XML request/response support -->
    <dependency>
        <groupId>com.fasterxml.jackson.dataformat</groupId>
        <artifactId>jackson-dataformat-xml</artifactId>
    </dependency>

    <!-- Testing -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

</dependencies>

<build>
    <plugins>

        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>

    </plugins>
</build>

</project>

```

5. Main application class

PostmanApiPracticeLabApplication.java

```

package com.example.postmanlab;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PostmanApiPracticeLabApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            PostmanApiPracticeLabApplication.class,
            args

```

```
    );  
  }  
}
```

What @SpringBootApplication does

It effectively combines the major Spring Boot application mechanisms:

```
@SpringBootConfiguration  
@EnableAutoConfiguration  
@ComponentScan
```

The application starts an embedded web server and exposes the REST endpoints.

6. Configuration

application.properties

```
spring.application.name=postman-api-practice-lab
```

```
server.port=8080
```

```
spring.servlet.multipart.max-file-size=10MB  
spring.servlet.multipart.max-request-size=20MB
```

```
spring.jackson.serialization.indent-output=true
```

```
server.error.include-message=always
```

```
logging.level.com.example.postmanlab=INFO
```

The base URL will therefore be:

```
http://localhost:8080
```

7. Product model

Product.java

```
package com.example.postmanlab.model;
```

```
public class Product {
```

```
    private Long id;  
    private String name;  
    private String category;
```

```
private Double price;
private Integer quantity;
private String description;

public Product() {
}

public Product(
    Long id,
    String name,
    String category,
    Double price,
    Integer quantity,
    String description) {

    this.id = id;
    this.name = name;
    this.category = category;
    this.price = price;
    this.quantity = quantity;
    this.description = description;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public Double getPrice() {
    return price;
}
```

```
public void setPrice(Double price) {
    this.price = price;
}

public Integer getQuantity() {
    return quantity;
}

public void setQuantity(Integer quantity) {
    this.quantity = quantity;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}
}
```

8. Product request DTO

ProductRequest.java

```
package com.example.postmanlab.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.PositiveOrZero;
import jakarta.validation.constraints.Size;

public class ProductRequest {

    @NotBlank(message = "Product name is required")
    @Size(min = 2, max = 100,
        message = "Product name must contain 2 to 100 characters")
    private String name;

    @NotBlank(message = "Category is required")
    private String category;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than zero")
    private Double price;
}
```

```
@NotNull(message = "Quantity is required")
@PositiveOrZero(message = "Quantity cannot be negative")
private Integer quantity;

@Size(max = 500,
      message = "Description cannot exceed 500 characters")
private String description;

public ProductRequest() {
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public Double getPrice() {
    return price;
}

public void setPrice(Double price) {
    this.price = price;
}

public Integer getQuantity() {
    return quantity;
}

public void setQuantity(Integer quantity) {
    this.quantity = quantity;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}
```

```
}
```

9. PATCH request DTO

ProductPatchRequest.java

PATCH differs from PUT because fields can be optional.

```
package com.example.postmanlab.dto;

import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.PositiveOrZero;
import jakarta.validation.constraints.Size;

public class ProductPatchRequest {

    @Size(min = 2, max = 100,
        message = "Product name must contain 2 to 100 characters")
    private String name;

    private String category;

    @Positive(message = "Price must be greater than zero")
    private Double price;

    @PositiveOrZero(message = "Quantity cannot be negative")
    private Integer quantity;

    @Size(max = 500,
        message = "Description cannot exceed 500 characters")
    private String description;

    public ProductPatchRequest() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
```

```
    this.category = category;
}

public Double getPrice() {
    return price;
}

public void setPrice(Double price) {
    this.price = price;
}

public Integer getQuantity() {
    return quantity;
}

public void setQuantity(Integer quantity) {
    this.quantity = quantity;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}
}
```

10. Product response DTO

ProductResponse.java

```
package com.example.postmanlab.dto;

public record ProductResponse(
    Long id,
    String name,
    String category,
    Double price,
    Integer quantity,
    String description
){
}
```

11. Login DTOs

LoginRequest.java

```
package com.example.postmanlab.dto;

import jakarta.validation.constraints.NotBlank;

public record LoginRequest(

    @NotBlank(message = "Username is required")
    String username,

    @NotBlank(message = "Password is required")
    String password

){
}
```

LoginResponse.java

```
package com.example.postmanlab.dto;

public record LoginResponse(
    String token,
    String tokenType,
    String message
){
}
```

12. Product service

ProductService.java

```
package com.example.postmanlab.service;

import com.example.postmanlab.dto.ProductPatchRequest;
import com.example.postmanlab.dto.ProductRequest;
import com.example.postmanlab.dto.ProductResponse;
import com.example.postmanlab.exception.ProductNotFoundException;
import com.example.postmanlab.model.Product;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

@Service
```

```

public class ProductService {

    private final Map<Long, Product> products =
        new ConcurrentHashMap<>();

    private final AtomicLong idGenerator =
        new AtomicLong(100);

    public ProductService() {

        createInternal(
            "Laptop",
            "Electronics",
            75000.0,
            10,
            "Business laptop"
        );

        createInternal(
            "Keyboard",
            "Electronics",
            2500.0,
            25,
            "Mechanical keyboard"
        );

        createInternal(
            "Office Chair",
            "Furniture",
            12000.0,
            15,
            "Ergonomic office chair"
        );

        createInternal(
            "Notebook",
            "Stationery",
            120.0,
            100,
            "A5 notebook"
        );
    }

    private Product createInternal(
        String name,
        String category,
        Double price,
        Integer quantity,
        String description) {

```

```

Long id = idGenerator.incrementAndGet();

Product product = new Product(
    id,
    name,
    category,
    price,
    quantity,
    description
);

products.put(id, product);

return product;
}

public List<ProductResponse> findAll(
    String category,
    Double minPrice,
    Double maxPrice,
    String name,
    int page,
    int size,
    String sortBy,
    String direction) {

    List<Product> filtered = new ArrayList<>(
        products.values()
    );

    if (category != null && !category.isBlank()) {

        filtered = filtered.stream()
            .filter(product ->
                product.getCategory()
                    .equalsIgnoreCase(category))
            .toList();
    }

    if (minPrice != null) {

        filtered = filtered.stream()
            .filter(product ->
                product.getPrice() >= minPrice)
            .toList();
    }

    if (maxPrice != null) {

        filtered = filtered.stream()

```

```

        .filter(product ->
            product.getPrice() <= maxPrice)
        .toList();
    }

    if (name != null && !name.isBlank()) {

        String searchName =
            name.toLowerCase(Locale.ROOT);

        filtered = filtered.stream()
            .filter(product ->
                product.getName()
                    .toLowerCase(Locale.ROOT)
                    .contains(searchName))
            .toList();
    }

    Comparator<Product> comparator;

    switch (sortBy.toLowerCase(Locale.ROOT)) {

        case "name":
            comparator =
                Comparator.comparing(Product::getName);
            break;

        case "price":
            comparator =
                Comparator.comparing(Product::getPrice);
            break;

        case "quantity":
            comparator =
                Comparator.comparing(Product::getQuantity);
            break;

        case "id":
        default:
            comparator =
                Comparator.comparing(Product::getId);
            break;
    }

    if ("desc".equalsIgnoreCase(direction)) {
        comparator = comparator.reversed();
    }

    filtered = filtered.stream()
        .sorted(comparator)

```

```

        .toList();

int fromIndex = page * size;

if (fromIndex >= filtered.size()) {
    return List.of();
}

int toIndex =
    Math.min(fromIndex + size, filtered.size());

return filtered.subList(fromIndex, toIndex)
    .stream()
    .map(this::toResponse)
    .toList();
}

public ProductResponse findById(Long id) {

    Product product = products.get(id);

    if (product == null) {
        throw new ProductNotFoundException(
            "Product not found with id: " + id
        );
    }

    return toResponse(product);
}

public ProductResponse create(ProductRequest request) {

    Long id = idGenerator.incrementAndGet();

    Product product = new Product(
        id,
        request.getName(),
        request.getCategory(),
        request.getPrice(),
        request.getQuantity(),
        request.getDescription()
    );

    products.put(id, product);

    return toResponse(product);
}

public ProductResponse replace(
    Long id,

```

```

        ProductRequest request) {

    if (!products.containsKey(id)) {

        throw new ProductNotFoundException(
            "Product not found with id: " + id
        );
    }

    Product product = new Product(
        id,
        request.getName(),
        request.getCategory(),
        request.getPrice(),
        request.getQuantity(),
        request.getDescription()
    );

    products.put(id, product);

    return toResponse(product);
}

public ProductResponse patch(
    Long id,
    ProductPatchRequest request) {

    Product product = products.get(id);

    if (product == null) {

        throw new ProductNotFoundException(
            "Product not found with id: " + id
        );
    }

    if (request.getName() != null) {
        product.setName(request.getName());
    }

    if (request.getCategory() != null) {
        product.setCategory(request.getCategory());
    }

    if (request.getPrice() != null) {
        product.setPrice(request.getPrice());
    }

    if (request.getQuantity() != null) {
        product.setQuantity(request.getQuantity());
    }

```

```

    }

    if (request.getDescription() != null) {
        product.setDescription(request.getDescription());
    }

    return toResponse(product);
}

public void delete(Long id) {

    Product removed = products.remove(id);

    if (removed == null) {

        throw new ProductNotFoundException(
            "Product not found with id: " + id
        );
    }
}

private ProductResponse toResponse(Product product) {

    return new ProductResponse(
        product.getId(),
        product.getName(),
        product.getCategory(),
        product.getPrice(),
        product.getQuantity(),
        product.getDescription()
    );
}
}

```

13. Product controller

ProductController.java

```

package com.example.postmanlab.controller;

import com.example.postmanlab.dto.ProductPatchRequest;
import com.example.postmanlab.dto.ProductRequest;
import com.example.postmanlab.dto.ProductResponse;
import com.example.postmanlab.service.ProductService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.Valid;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;

```

```

import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.UUID;

@RestController
@RequestMapping("/api/v1/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    /**
     * GET /api/v1/products
     *
     * Demonstrates:
     * Query parameters
     * Filtering
     * Pagination
     * Sorting
     */
    @GetMapping
    public ResponseEntity<List<ProductResponse>> findAll(

        @RequestParam(required = false)
        String category,

        @RequestParam(required = false)
        Double minPrice,

        @RequestParam(required = false)
        Double maxPrice,

        @RequestParam(required = false)
        String name,

        @RequestParam(defaultValue = "0")
        int page,

        @RequestParam(defaultValue = "10")
        int size,

        @RequestParam(defaultValue = "id")
        String sortBy,

```

```

    @RequestParam(defaultValue = "asc")
    String direction,

    @RequestHeader(
        value = "X-Request-ID",
        required = false)
    String requestId) {

    if (page < 0) {
        throw new IllegalArgumentException(
            "page cannot be negative"
        );
    }

    if (size < 1 || size > 100) {
        throw new IllegalArgumentException(
            "size must be between 1 and 100"
        );
    }

    List<ProductResponse> products =
        productService.findAll(
            category,
            minPrice,
            maxPrice,
            name,
            page,
            size,
            sortBy,
            direction
        );

    String generatedRequestId =
        requestId != null
            ? requestId
            : UUID.randomUUID().toString();

    return ResponseEntity.ok()
        .header("X-Request-ID", generatedRequestId)
        .body(products);
}

/*
 * GET /api/v1/products/{id}
 *
 * Demonstrates path variable.
 */
@GetMapping(
    value =("/{id}",
    produces = {

```

```

        MediaType.APPLICATION_JSON_VALUE,
        MediaType.APPLICATION_XML_VALUE
    }
}

public ProductResponse findById(
    @PathVariable Long id) {

    return productService.findById(id);
}

/*
 * POST
 *
 * Demonstrates JSON request body.
 */
@PostMapping(
    consumes = MediaType.APPLICATION_JSON_VALUE,
    produces = MediaType.APPLICATION_JSON_VALUE
)
public ResponseEntity<ProductResponse> create(
    @Valid @RequestBody ProductRequest request) {

    ProductResponse response =
        productService.create(request);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(response);
}

/*
 * POST XML
 *
 * Same business operation,
 * different request content type.
 */
@PostMapping(
    value = "/xml",
    consumes = MediaType.APPLICATION_XML_VALUE,
    produces = MediaType.APPLICATION_XML_VALUE
)
public ResponseEntity<ProductResponse> createXml(
    @Valid @RequestBody ProductRequest request) {

    ProductResponse response =
        productService.create(request);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(response);
}

```

```

}

/*
 * PUT
 *
 * Complete replacement.
 */
@PutMapping(
    value =("/{id}",
    consumes = MediaType.APPLICATION_JSON_VALUE
)
public ProductResponse replace(
    @PathVariable Long id,
    @Valid @RequestBody ProductRequest request) {

    return productService.replace(id, request);
}

/*
 * PATCH
 *
 * Partial modification.
 */
@PatchMapping(
    value =("/{id}",
    consumes = MediaType.APPLICATION_JSON_VALUE
)
public ProductResponse patch(
    @PathVariable Long id,
    @Valid @RequestBody ProductPatchRequest request) {

    return productService.patch(id, request);
}

/*
 * DELETE
 */
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(
    @PathVariable Long id) {

    productService.delete(id);

    return ResponseEntity.noContent().build();
}

/*
 * HEAD
 *
 * Returns headers without a response body.

```

```

    */
    @RequestMapping(
        value =("/{id}",
        method = RequestMethod.HEAD
    )
    public ResponseEntity<Void> head(
        @PathVariable Long id) {

        productService.findById(id);

        return ResponseEntity.ok()
            .header(
                HttpHeaders.CONTENT_TYPE,
                MediaType.APPLICATION_JSON_VALUE
            )
            .build();
    }

    /*
    * OPTIONS
    */
    @RequestMapping(
        value =("/{id}",
        method = RequestMethod.OPTIONS
    )
    public ResponseEntity<Void> options() {

        return ResponseEntity.ok()
            .allow(
                RequestMethod.GET,
                RequestMethod.POST,
                RequestMethod.PUT,
                RequestMethod.PATCH,
                RequestMethod.DELETE,
                RequestMethod.HEAD,
                RequestMethod.OPTIONS
            )
            .build();
    }
}

```

14. Authentication service

This is deliberately a **learning/demo authentication system**, not production authentication.

AuthService.java

```
package com.example.postmanlab.service;
```

```

import org.springframework.stereotype.Service;

import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

@Service
public class AuthService {

    private final Map<String, String> tokens =
        new ConcurrentHashMap<>();

    public String login(
        String username,
        String password) {

        if (!"admin".equals(username)
            || !"admin123".equals(password)) {

            throw new IllegalArgumentException(
                "Invalid username or password"
            );
        }

        String token =
            "demo-" + UUID.randomUUID();

        tokens.put(token, username);

        return token;
    }

    public boolean isValid(String token) {

        return token != null
            && tokens.containsKey(token);
    }

    public String getUsername(String token) {

        return tokens.get(token);
    }
}

```

15. Authentication controller

AuthController.java

```

package com.example.postmanlab.controller;

import com.example.postmanlab.dto.LoginRequest;
import com.example.postmanlab.dto.LoginResponse;
import com.example.postmanlab.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(
        @Valid @RequestBody LoginRequest request) {

        String token =
            authService.login(
                request.username(),
                request.password()
            );

        return ResponseEntity.ok(
            new LoginResponse(
                token,
                "Bearer",
                "Login successful"
            )
        );
    }

    @GetMapping("/profile")
    public ResponseEntity<?> profile(
        @RequestHeader(
            value = "Authorization",
            required = false)
        String authorization) {

        if (authorization == null
            || !authorization.startsWith("Bearer ")) {

            return ResponseEntity

```

```

        .status(HttpStatus.UNAUTHORIZED)
        .body(
            Map.of(
                "status", 401,
                "message",
                "Authorization header required"
            )
        );
    }

    String token =
        authorization.substring(7);

    if (!authService.isValid(token)) {

        return ResponseEntity
            .status(HttpStatus.UNAUTHORIZED)
            .body(
                Map.of(
                    "status", 401,
                    "message",
                    "Invalid or expired token"
                )
            );
    }

    return ResponseEntity.ok(
        Map.of(
            "username",
            authService.getUsername(token),
            "message",
            "Protected resource accessed successfully"
        )
    );
}
}
}

```

16. Utility controller

This controller demonstrates several request types that are very useful when learning Postman.

UtilityController.java

```

package com.example.postmanlab.controller;

import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.MediaType;

```

```

import org.springframework.http.ResponseEntity;
import org.springframework.util.MultiValueMap;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.util.*;

@RestController
@RequestMapping("/api/v1")
public class UtilityController {

    /**
     * Simple GET
     */
    @GetMapping("/hello")
    public String hello(
        @RequestParam(defaultValue = "World")
        String name) {

        return "Hello, " + name + "!";
    }

    /**
     * Plain text request/response
     */
    @PostMapping(
        value = "/echo-text",
        consumes = MediaType.TEXT_PLAIN_VALUE,
        produces = MediaType.TEXT_PLAIN_VALUE
    )
    public String echoText(
        @RequestBody String text) {

        return "Server received: " + text;
    }

    /**
     * application/x-www-form-urlencoded
     */
    @PostMapping(
        value = "/form-login",
        consumes =
            MediaType.APPLICATION_FORM_URLENCODED_VALUE
    )
    public ResponseEntity<?> formLogin(
        @RequestParam String username,
        @RequestParam String password) {

        boolean valid =

```

```

        "admin".equals(username)
        && "admin123".equals(password);

    if (!valid) {

        return ResponseEntity
            .status(401)
            .body(
                Map.of(
                    "success", false,
                    "message",
                    "Invalid credentials"
                )
            );
    }

    return ResponseEntity.ok(
        Map.of(
            "success", true,
            "message",
            "Form login successful",
            "username",
            username
        )
    );
}

/*
 * Read multiple form parameters.
 */
@PostMapping(
    value = "/form-data",
    consumes =
        MediaType.APPLICATION_FORM_URLENCODED_VALUE
)
public Map<String, Object> formData(
    @RequestParam MultiValueMap<String, String> form) {

    return Map.of(
        "received", form
    );
}

/*
 * Multipart file upload.
 */
@PostMapping(
    value = "/upload",
    consumes = MediaType.MULTIPART_FORM_DATA_VALUE
)

```

```

public ResponseEntity<?> upload(
    @RequestPart("file")
    MultipartFile file,

    @RequestPart(
        value = "description",
        required = false)
    String description) throws IOException {

    return ResponseEntity.ok(
        Map.of(
            "fileName",
            file.getOriginalFilename(),
            "contentType",
            file.getContentType(),
            "size",
            file.getSize(),
            "description",
            description == null
                ? ""
                : description,
            "message",
            "File uploaded successfully"
        )
    );
}

/*
 * Request headers.
 */
@GetMapping("/headers")
public Map<String, Object> headers(
    @RequestHeader(
        value = "X-API-KEY",
        required = false)
    String apiKey,

    @RequestHeader(
        value = "X-Client-Version",
        required = false)
    String clientVersion,

    @RequestHeader(
        value = "User-Agent",
        required = false)
    String userAgent) {

    return Map.of(
        "xApiKey",
        apiKey == null ? "" : apiKey,

```

```

        "clientVersion",
        clientVersion == null
            ? ""
            : clientVersion,
        "userAgent",
        userAgent == null
            ? ""
            : userAgent
    );
}

/*
 * Cookie demonstration.
 */
@GetMapping("/cookies")
public Map<String, Object> cookies(
    @CookieValue(
        value = "clientId",
        required = false)
    String clientId,

    @CookieValue(
        value = "theme",
        required = false)
    String theme) {

    return Map.of(
        "clientId",
        clientId == null ? "" : clientId,
        "theme",
        theme == null ? "" : theme
    );
}

/*
 * Set cookies.
 */
@PostMapping("/cookies")
public ResponseEntity<?> setCookies() {

    ResponseEntity.BodyBuilder response =
        ResponseEntity.ok();

    return response
        .header(
            "Set-Cookie",
            "clientId=client-123; Path=/"
        )
        .header(
            "Set-Cookie",

```

```

        "theme=dark; Path=/"
    )
    .body(
        Map.of(
            "message",
            "Cookies have been set"
        )
    );
}

/*
 * Inspect almost everything received by the server.
 */
@PostMapping("/echo")
public Map<String, Object> echo(
    HttpServletRequest request,
    @RequestParam Map<String, String> params,
    @RequestBody(required = false) String body) {

    Map<String, String> headers =
        new LinkedHashMap<>();

    Enumeration<String> headerNames =
        request.getHeaderNames();

    if (headerNames != null) {

        while (headerNames.hasMoreElements()) {

            String name =
                headerNames.nextElement();

            headers.put(
                name,
                request.getHeader(name)
            );
        }
    }

    return Map.of(
        "method",
        request.getMethod(),

        "uri",
        request.getRequestURI(),

        "queryParameters",
        params,

        "headers",

```

```

        headers,

        "body",
        body == null ? "" : body
    );
}

/*
 * Health check.
 */
@GetMapping("/health")
public Map<String, Object> health() {

    return Map.of(
        "status",
        "UP",

        "application",
        "postman-api-practice-lab",

        "timestamp",
        new Date().toString()
    );
}

/*
 * Deliberately returns 201.
 */
@PostMapping("/status/created")
public ResponseEntity<?> created() {

    return ResponseEntity
        .status(201)
        .body(
            Map.of(
                "status",
                201,
                "message",
                "Resource created"
            )
        );
}

/*
 * Deliberately returns 204.
 */
@DeleteMapping("/status/no-content")
public ResponseEntity<Void> noContent() {

    return ResponseEntity.noContent().build();
}

```

```

}

/*
 * Deliberately returns 400.
 */
@GetMapping("/status/bad-request")
public ResponseEntity<?> badRequest() {

    return ResponseEntity
        .badRequest()
        .body(
            Map.of(
                "status", 400,
                "message",
                "This is a demonstration of HTTP 400"
            )
        );
}

```

```

/*
 * Deliberately returns 401.
 */
@GetMapping("/status/unauthorized")
public ResponseEntity<?> unauthorized() {

    return ResponseEntity
        .status(401)
        .body(
            Map.of(
                "status", 401,
                "message",
                "Authentication required"
            )
        );
}

```

```

/*
 * Deliberately returns 404.
 */
@GetMapping("/status/not-found")
public ResponseEntity<?> notFound() {

    return ResponseEntity
        .status(404)
        .body(
            Map.of(
                "status", 404,
                "message",
                "Resource not found"
            )
        );
}

```

```
    );  
}  
}
```

17. Custom exception

ProductNotFoundException.java

```
package com.example.postmanlab.exception;  
  
public class ProductNotFoundException  
    extends RuntimeException {  
  
    public ProductNotFoundException(String message) {  
        super(message);  
    }  
}
```

18. API error object

ApiError.java

```
package com.example.postmanlab.exception;  
  
import java.time.LocalDateTime;  
import java.util.Map;  
  
public record ApiError(  
    LocalDateTime timestamp,  
    int status,  
    String error,  
    String message,  
    String path,  
    Map<String, String> validationErrors  
) {  
}
```

19. Global exception handler

GlobalExceptionHandler.java

```
package com.example.postmanlab.exception;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ApiError> handleProductNotFound(
        ProductNotFoundException exception,
        HttpServletRequest request) {

        ApiError error =
            new ApiError(
                LocalDateTime.now(),
                HttpStatus.NOT_FOUND.value(),
                "NOT_FOUND",
                exception.getMessage(),
                request.getRequestURI(),
                Map.of()
            );

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(
        MethodArgumentNotValidException exception,
        HttpServletRequest request) {

        Map<String, String> validationErrors =
            new LinkedHashMap<>();

        exception.getBindingResult()
            .getFieldErrors()
            .forEach(error ->
                validationErrors.put(
```

```

        error.getField(),
        error.getDefaultMessage()
    )
};

ApiError error =
    new ApiError(
        LocalDateTime.now(),
        HttpStatus.BAD_REQUEST.value(),
        "VALIDATION_ERROR",
        "Request validation failed",
        request.getRequestURI(),
        validationErrors
    );

return ResponseEntity
    .badRequest()
    .body(error);
}

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<ApiError> handleBadRequest(
    IllegalArgumentException exception,
    HttpServletRequest request) {

    ApiError error =
        new ApiError(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            "BAD_REQUEST",
            exception.getMessage(),
            request.getRequestURI(),
            Map.of()
        );

    return ResponseEntity
        .badRequest()
        .body(error);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiError> handleGeneralException(
    Exception exception,
    HttpServletRequest request) {

    ApiError error =
        new ApiError(
            LocalDateTime.now(),
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "INTERNAL_SERVER_ERROR",

```

```
        exception.getMessage(),
        request.getRequestURI(),
        Map.of()
    );

    return ResponseEntity
        .status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(error);
    }
}
```

20. Test class

PostmanApiPracticeLabApplicationTests.java

```
package com.example.postmanlab;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class PostmanApiPracticeLabApplicationTests {

    @Test
    void contextLoads() {
    }
}
```

21. Build and run

From the project directory:

```
mvn clean install
```

Then:

```
mvn spring-boot:run
```

Or:

```
java -jar target/postman-api-practice-lab-1.0.0.jar
```

Spring Boot creates an executable JAR that can be run using java -jar. [Home](#)

You should see the application running on:

```
http://localhost:8080
```

22. First Postman request

Create a Postman request:

GET `http://localhost:8080/api/v1/hello`

Response:

Hello, World!

Now:

GET `http://localhost:8080/api/v1/hello?name=Anurag`

Response:

Hello, Anurag!

23. Understanding URL parameters

There are two major parameter types you should distinguish.

Path parameter

GET `/api/v1/products/101`

Here:

101

is a **path variable**.

Java:

```
@GetMapping("/{id}")  
public ProductResponse findById(  
    @PathVariable Long id)
```

Query parameter

GET `/api/v1/products?category=Electronics`

Here:

category=Electronics

is a query parameter.

Java:

@RequestParam String category

24. GET all products

Postman:

GET <http://localhost:8080/api/v1/products>

Example response:

```
[
  {
    "id": 101,
    "name": "Laptop",
    "category": "Electronics",
    "price": 75000.0,
    "quantity": 10,
    "description": "Business laptop"
  },
  {
    "id": 102,
    "name": "Keyboard",
    "category": "Electronics",
    "price": 2500.0,
    "quantity": 25,
    "description": "Mechanical keyboard"
  }
]
```

25. GET with filtering

GET <http://localhost:8080/api/v1/products?category=Electronics>

Multiple filters:

GET

<http://localhost:8080/api/v1/products?category=Electronics&minPrice=2000&maxPrice=80000>

Name search:

GET <http://localhost:8080/api/v1/products?name=lap>

26. Pagination

GET <http://localhost:8080/api/v1/products?page=0&size=2>

Page 2:

GET <http://localhost:8080/api/v1/products?page=1&size=2>

The calculation is:

$\text{fromIndex} = \text{page} \times \text{size}$

For:

page = 1
size = 2

we get:

$\text{fromIndex} = 1 \times 2$
 $= 2$

27. Sorting

Sort by price:

GET <http://localhost:8080/api/v1/products?sortBy=price>

Descending:

GET <http://localhost:8080/api/v1/products?sortBy=price&direction=desc>

Sort by name:

GET <http://localhost:8080/api/v1/products?sortBy=name&direction=asc>

28. POST — Create product

Postman:

POST <http://localhost:8080/api/v1/products>

Select:

Body
→ raw
→ JSON

Body:

```
{  
  "name": "Monitor",  
  "category": "Electronics",  
  "price": 18000,  
  "quantity": 20,  
  "description": "27 inch monitor"  
}
```

Header:

Content-Type: application/json

Response:

```
{  
  "id": 105,  
  "name": "Monitor",  
  "category": "Electronics",  
  "price": 18000.0,  
  "quantity": 20,  
  "description": "27 inch monitor"  
}
```

Status:

201 Created

29. Understanding Content-Type

Content-Type tells the server what format the **request body** contains.

Examples:

application/json

means:

```
{  
  "name": "Monitor"  
}
```

Whereas:

application/xml

means:

```
<ProductRequest>
  <name>Monitor</name>
</ProductRequest>
```

Other important values:

Content-Type	Purpose
application/json	JSON
application/xml	XML
text/plain	Plain text
application/x-www-form-urlencoded	Form parameters
multipart/form-data	File/form upload

30. PUT — Complete replacement

PUT http://localhost:8080/api/v1/products/101

Body:

```
{
  "name": "Gaming Laptop",
  "category": "Electronics",
  "price": 95000,
  "quantity": 8,
  "description": "High-performance gaming laptop"
}
```

PUT means conceptually:

Replace the representation of resource 101 with this new representation.

31. PATCH — Partial update

PATCH <http://localhost:8080/api/v1/products/101>

Body:

```
{  
  "price": 90000  
}
```

Only price changes.

The other fields remain unchanged.

This is one of the most important practical differences:

PUT	PATCH
Complete replacement	Partial modification
Usually sends complete resource	Sends changed fields
All required fields normally supplied	Only changed fields supplied

32. DELETE

DELETE <http://localhost:8080/api/v1/products/101>

Expected status:

204 No Content

33. XML API

Create an XML product:

POST <http://localhost:8080/api/v1/products/xml>

Headers:

Content-Type: application/xml

Accept: application/xml

Body:

```
<ProductRequest>
  <name>XML Monitor</name>
  <category>Electronics</category>
  <price>22000</price>
  <quantity>5</quantity>
  <description>Monitor submitted as XML</description>
</ProductRequest>
```

This demonstrates:

Request:
XML

Response:
XML

34. Accept versus Content-Type

This distinction is extremely important in Postman.

Content-Type

Answers:

What format am I sending?

Example:

Content-Type: application/json

Accept

Answers:

What response format do I want?

Example:

Accept: application/xml

Therefore:

Content-Type = request format

Accept = desired response format

35. Plain text API

Postman:

POST <http://localhost:8080/api/v1/echo-text>

Header:

Content-Type: text/plain

Body:

This is a plain text message.

Response:

Server received: This is a plain text message.

36. Form URL Encoded API

Postman:

POST <http://localhost:8080/api/v1/form-login>

Choose:

Body

→ x-www-form-urlencoded

Add:

KEY	VALUE
username	admin
password	admin123

This produces a request conceptually similar to:

username=admin&password=admin123

37. Multipart file upload

Postman:

POST <http://localhost:8080/api/v1/upload>

Select:

Body

→ form-data

Add:

KEY	TYPE	VALUE
file	File	Select any file
description	Text	Resume document

Do **not** manually set:

Content-Type: multipart/form-data

Postman should generate the appropriate multipart boundary.

38. Headers API

Request:

GET <http://localhost:8080/api/v1/headers>

Headers:

X-API-KEY: ABC123

X-Client-Version: 1.0

The server also receives the normal Postman-generated headers.

Example response:

```
{
  "xApiKey": "ABC123",
  "clientVersion": "1.0",
  "userAgent": "PostmanRuntime/..."
}
```

39. Cookies

First:

POST <http://localhost:8080/api/v1/cookies>

The response sets:

clientId=client-123
theme=dark

Then:

GET <http://localhost:8080/api/v1/cookies>

Response:

```
{  
  "clientId": "client-123",  
  "theme": "dark"  
}
```

40. Authentication flow

This is one of the most useful Postman workflows.

Step 1 — Login

POST <http://localhost:8080/api/v1/auth/login>

Body:

```
{  
  "username": "admin",  
  "password": "admin123"  
}
```

Response:

```
{  
  "token": "demo-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx",  
  "tokenType": "Bearer",  
  "message": "Login successful"  
}
```

Copy the token.

41. Calling protected API

Request:

GET http://localhost:8080/api/v1/auth/profile

Header:

Authorization: Bearer demo-xxxxxxx...

Response:

```
{
  "username": "admin",
  "message": "Protected resource accessed successfully"
}
```

Without the header:

401 Unauthorized

42. Bearer token concept

The standard form is:

Authorization: Bearer <token>

For example:

Authorization: Bearer abc123

The word:

Bearer

indicates the authentication scheme.

This project uses a deliberately simple in-memory token for learning. Real applications should use a properly designed authentication system such as OAuth 2.0/OIDC or Spring Security rather than this demo implementation.

43. Validation test

Send an invalid POST:

POST <http://localhost:8080/api/v1/products>

Body:

```
{
  "name": "",
  "category": "",
  "price": -10,
  "quantity": -5,
  "description": "Invalid product"
}
```

Response:

```
{
  "timestamp": "2026-08-18T04:30:00",
  "status": 400,
  "error": "VALIDATION_ERROR",
  "message": "Request validation failed",
  "path": "/api/v1/products",
  "validationErrors": {
    "name": "Product name is required",
    "category": "Category is required",
    "price": "Price must be greater than zero",
    "quantity": "Quantity cannot be negative"
  }
}
```

44. 404 test

Request:

GET <http://localhost:8080/api/v1/products/999999>

Response:

```
{
  "timestamp": "2026-08-18T04:30:00",
  "status": 404,
  "error": "NOT_FOUND",
  "message": "Product not found with id: 999999",
  "path": "/api/v1/products/999999",
  "validationErrors": {}
}
```

45. HTTP methods covered

The project demonstrates:

GET
POST
PUT
PATCH
DELETE
HEAD
OPTIONS

Their basic purpose:

Method	Purpose
GET	Read resource
POST	Create/process resource
PUT	Replace resource
PATCH	Partially modify resource
DELETE	Delete resource
HEAD	Get headers without body
OPTIONS	Discover supported operations

46. Complete API catalog

Method	URL	Main concept
GET	/api/v1/hello	Query parameter
GET	/api/v1/health	Health API
GET	/api/v1/products	Query/filter/pagination
GET	/api/v1/products/{id}	Path variable
POST	/api/v1/products	JSON

POST	/api/v1/products/xml	XML
PUT	/api/v1/products/{id}	Full update
PATCH	/api/v1/products/{id}	Partial update
DELETE	/api/v1/products/{id}	Delete
HEAD	/api/v1/products/{id}	HEAD
OPTIONS	/api/v1/products/{id}	OPTIONS
POST	/api/v1/auth/login	Login
GET	/api/v1/auth/profile	Bearer token
POST	/api/v1/form-login	Form URL encoded
POST	/api/v1/form-data	Multiple form values
POST	/api/v1/upload	Multipart
GET	/api/v1/headers	Request headers
POST	/api/v1/cookies	Set cookies
GET	/api/v1/cookies	Read cookies
POST	/api/v1/echo-text	Plain text
POST	/api/v1/echo	Request inspection
POST	/api/v1/status/created	201
DELETE	/api/v1/status/no-content	204
GET	/api/v1/status/bad-request	400
GET	/api/v1/status/unauthorized	401
GET	/api/v1/status/not-found	404

47. Postman variables

Instead of repeatedly writing:

`http://localhost:8080`

create a Postman environment variable:

`baseUrl`

with value:

`http://localhost:8080`

Then use:

`{{baseUrl}}/api/v1/products`

This is extremely important for real-world Postman projects.

For example:

Development:

`http://localhost:8080`

Testing:

`https://test.example.com`

Production:

`https://api.example.com`

The request remains:

`{{baseUrl}}/api/v1/products`

Only the environment changes.

48. Recommended Postman environment

Create:

Postman API Lab

Variables:

Variable	Initial value	Current value
baseUrl	http://localhost:8080	http://localhost:8080
token	empty	empty
productId	101	101
requestId	postman-001	postman-001

49. Automatically save login token

In the **Tests** section of the login request:

```
const json = pm.response.json();  
  
if (json.token) {  
  pm.environment.set("token", json.token);  
}
```

Then protected requests can use:

Authorization: Bearer {{token}}

This is much better than manually copying tokens.

50. Automatically save product ID

For the POST product request:

```
const json = pm.response.json();  
  
if (json.id) {  
  pm.environment.set("productId", json.id);  
}
```

Then:

GET {{baseUrl}}/api/v1/products/{{productId}}

will automatically use the newly created product.

51. Postman test examples

For a successful POST:

```
pm.test("Status code is 201", function () {  
  pm.response.to.have.status(201);  
});
```

Check response JSON:

```
pm.test("Product has an ID", function () {  
  
  const json = pm.response.json();  
  
  pm.expect(json.id).to.exist;  
});
```

Check product name:

```
pm.test("Product name is correct", function () {  
  
  const json = pm.response.json();  
  
  pm.expect(json.name).to.eql("Monitor");  
});
```

52. GET API test

```
pm.test("Status code is 200", function () {  
  pm.response.to.have.status(200);  
});
```

```
pm.test("Response is JSON", function () {  
  
  pm.expect(  
    pm.response.headers  
      .get("Content-Type")  
  ).to.include("application/json");  
});
```

```
pm.test("Response is an array", function () {  
  
  const json = pm.response.json();  
  
  pm.expect(json).to.be.an("array");  
});
```

53. Validation test

```
pm.test("Validation returns 400", function () {  
  pm.response.to.have.status(400);  
});
```

```
pm.test("Validation error exists", function () {  
  
  const json = pm.response.json();  
  
  pm.expect(json.error)  
    .to.eql("VALIDATION_ERROR");  
});
```

54. Postman request lifecycle

A very important concept is:

Pre-request Script

|

v

HTTP Request

|

v

Spring Boot

|

v

Controller

|

v

Service

|

v

Response

|

v

Postman Tests

In Postman:

Pre-request Script

runs before the API request.

Tests

run after the API response.

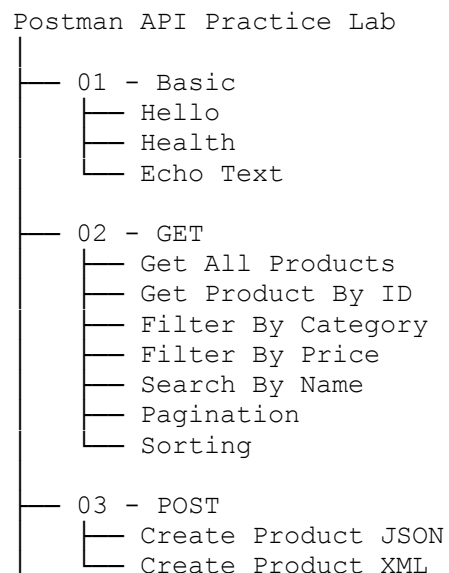
55. Complete CRUD workflow in Postman

The most useful sequence for this project is:

1. Health Check
 - ↓
 2. Login
 - ↓
 3. Create Product
 - ↓
 4. Get Product
 - ↓
 5. Get All Products
 - ↓
 6. Filter Products
 - ↓
 7. Sort Products
 - ↓
 8. Update Product using PUT
 - ↓
 9. Partially update using PATCH
 - ↓
 10. Delete Product
 - ↓
 11. Verify Deleted Product
-

56. Recommended Postman collection structure

Create a collection:



04 - PUT	Replace Product
05 - PATCH	Partial Update
06 - DELETE	Delete Product
07 - Authentication	Login Get Profile
08 - Headers	Request Headers
09 - Cookies	Set Cookies Read Cookies
10 - Forms	Form Login Form Data
11 - Multipart	File Upload
12 - Status Codes	201 Created 204 No Content 400 Bad Request 401 Unauthorized 404 Not Found
13 - Tests	CRUD Tests Validation Tests Authentication Tests

57. Important API terminology

Term	Meaning
API	Interface through which applications communicate
REST	Architectural style for HTTP-based APIs
Endpoint	Specific API URL
URI	Resource identifier

URL	Address used to locate/access a resource
HTTP method	Operation such as GET/POST/PUT/DELETE
Request	Message sent by client
Response	Message returned by server
Header	Metadata associated with request/response
Body	Main data carried by request/response
Parameter	Input supplied to an API
Path variable	Parameter embedded in URL path
Query parameter	Parameter after ?
Request body	Data supplied in body
Content-Type	Format of request body
Accept	Desired response format
Cookie	Client-side HTTP state data
Token	Credential representing authentication
Status code	Numeric result of HTTP operation
JSON	JavaScript Object Notation
XML	Extensible Markup Language
Multipart	Request format commonly used for file upload
DTO	Data Transfer Object
CRUD	Create, Read, Update, Delete

58. The most important distinction: parameter types

Consider this URL:

GET /api/v1/products/101?category=Electronics&sortBy=price

It contains:

101

→ Path variable

category=Electronics

→ Query parameter

sortBy=price

→ Query parameter

And a header could additionally be:

X-API-KEY: ABC123

while a cookie could be:

clientId=123

So one HTTP request can contain:

URL

- └ Path
- └ Query parameters

Headers

- └ Authorization
- └ Content-Type
- └ Accept
- └ Custom headers

Cookies

- └ clientId

Body

- └ JSON/XML/Form/Multipart

That is exactly why this project is useful for Postman practice.

59. Example of a complete request

For example:

PATCH

http://localhost:8080/api/v1/products/101?source=postman

Headers:

Authorization: Bearer {{token}}
Content-Type: application/json
Accept: application/json
X-Request-ID: {{requestId}}

Cookie:

clientId=client-123

Body:

```
{  
  "price": 88000,  
  "quantity": 15  
}
```

The server can therefore receive:

HTTP method



PATCH

Path variable



101

Query parameter



source=postman

Headers



Authorization
Content-Type
Accept
X-Request-ID

Cookie



clientId

Body

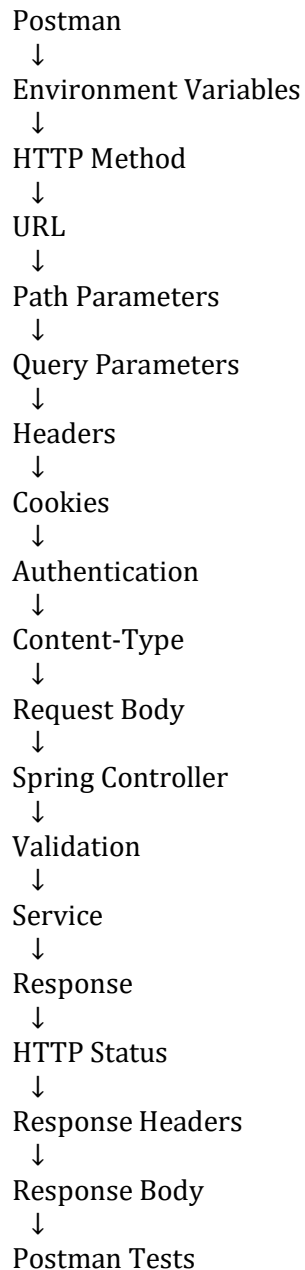


price
quantity

This is a very realistic representation of what you encounter in enterprise APIs.

60. What this project teaches you

After completing this project in Postman, you will understand the complete flow:



And you will have practical examples for virtually all of the **common HTTP/REST request patterns used when testing APIs in Postman**.

One important distinction: there is no literal finite set called “every kind of API.” REST APIs can use many application-specific conventions and protocols. The project above covers the major HTTP

methods and input/output mechanisms you will routinely encounter in real-world REST API testing, while keeping the backend small enough to understand end-to-end.